

Load Distribution

يعني:

توزيع الضغط والطلبات على عدة Workers أو Servers بدال ما نحط الحمل كله على جزء واحد

ليش محتاجه؟

تخيل عندك متجر إلكتروني. وفجأة دخل ١٠٠٠٠ مستخدم بلحظة وحدة

بدون Load Distribution

كل الطلبات بتروح إلى:

- Process واحد
- أو Server واحد

النتيجة

CPU يصل ١٠٠%
بطء شديد
Crash

الحل نوزع الطلبات.

بدل:

كل الحمل على عامل واحد

بتصير :

عامل ١ → جزء من الطلبات
عامل ٢ → جزء آخر
عامل ٣ → جزء آخر

.

هلق خلينا نروح عال تطبيق عالكود عنا

app/Services/LoadBalancerService.php

```
private const SERVERS = [
    ['id' => 'server-1', 'host' => '10.0.0.1', 'port' => 8001,
    'weight' => 3, 'zone' => 'primary'],
    ['id' => 'server-2', 'host' => '10.0.0.2', 'port' => 8002,
    'weight' => 2, 'zone' => 'primary'],
    ['id' => 'server-3', 'host' => '10.0.0.3', 'port' => 8003,
    'weight' => 1, 'zone' => 'secondary'],
];
```

هون في عنا كذا نقطة بدها شرح

عنا اوزان مختلفة لبييش؟؟

لا قبل نسيت قلكم انو هون نحنا عم نعرف الخوادم

جواب السؤال انو عنا اوزان مختلفة مشان نخلي سيرفر يقوم بشغل اكثر من سيرفر تاني

يعني هلق عنا ٣ و ٢ و ١

مجموعن ٦

يعني القدرة الكلية هوة ٧ يعني ٦ هية ١٠٠٪ من قدرة السيرفر

يعني مثلا قدرة السيرفر الأول هية وزنو على ٧ يعني ٦/٣ يعني ٥٠ بالمية يعني بيتحمل ضغط اكثر شي فهو سيرفر أساسي

Weighted Round Robin هي الطريقة اللي استعملناها للتقسيم اسمها

الجزء الثاني :

Redis Atomic Counter

```
private const COUNTER_KEY = 'lb:round_robin_counter';
private const METRICS_KEY = 'lb:server_metrics';
private const HEALTH_KEY = 'lb:server_health';
```

يخزن رقم يتزايد في كل طلب منستخدمه : 'lb:round_robin_counter'
لنختار أيا سيرفر بدنا نستهدمه

يخزن عدد الطلبات التي استقبلها كل سيرفر فائدته مشان 'lb:server_metrics' المراقبة و هالامور :

يخزن حالة السيرفر شغال او مطفي يعني فائدته انو 'lb:server_health' ما نرسل طلب لسيرفر مطفي:

```
$counter = Cache::getRedis()->incr(self::COUNTER_KEY);  
$index = ($counter - 1) % count($weightedPool);
```

يزيد العداد عند كل ريكوريسيت REDIS عم نخلي

بس الفكرة انو هاد مو متغير عادي مثل أيا متغير بل :

- مخزن في Redis
- مشترك بين كل السيرفرات
- آمن من التكرار (Atomic)

`count($weightedPool)`

يعطيك عدد العناصر في القائمة.

مثال :

[server-1, server-1, server-1, server-2, server-2, server-3]

عددها = 6

2. (\$counter - 1)

لأن العد يبدأ من ١، ونريد تحويله إلى ٠-based index-

counter	index
1	0
2	1
3	2

3. % (modulus)

هذا أهم شيء:

يرجع الرقم ضمن حدود المصفوفة

مثال كامل

نفترض:

weightedPool size = 6

الحالات:

counter = 1

$(1 - 1) \% 6 = 0$

→ server-1

counter = 2

$(2 - 1) \% 6 = 1$

→ server-1

counter = 4

$(4 - 1) \% 6 = 3$

→ server-2

counter = 6

$(6 - 1) \% 6 = 5$

→ server-3

counter = 7

$(7 - 1) \% 6 = 0$

→ يرجع للبداية

ماذا يعني هذا عملياً؟

هذا الكود يحقق:

Round Robin Loop

يعني: يدور على السيرفرات بشكل دائري

هلق عنا فقرة ال FAILOVER

```
// التحقق من صحة الخادم المختار
if (!$this->isHealthy($server['id'])) {
    // Failover: جرب الخادم التالي في الدورة
    $server = $this->failover($server['id'], $weightedPool,
$index);
}

// تتبع عدد الطلبات لكل خادم في Redis
Cache::getRedis()->hincrby(self::METRICS_KEY, $server['id'],
1);
```

يعني منشوف اذا السيرفر مو شغال منجرب الخادم الثاني يلي بعده ببساطة

مثال لتفهمو ال FAILOVER

عنا ٣ كاشيرات

- كاشير ١ موجود
- كاشير ٢ مريض
- كاشير ٣ موجود

النظام شو بيعمل ؟

لا يرسل زبائن للكاشير المريض
يحولهم تلقائيا لكاشير شغال

عنا كونترولر لنجرب كلشي حكيانه حطلكم ياه واضح مابدو شرح اللي بدو
شرح حاترك تعليقات جنب الكود

```

class LoadBalancerController extends Controller
{
    public function __construct(
        private readonly LoadBalancerService $lb
    ) {}

    public function health(): JsonResponse
    {
        return response()->json([
            'status' => 'ok',
            'servers' => $this->lb->healthCheck(),
        ]);
    }

    // POST /api/lb/simulate - طلب N محاكاة توزيع
    public function simulate(Request $request): JsonResponse
    {
        $count = (int) $request->input('requests', 100);
        $count = min(max($count, 1), 1000);

        $result = $this->lb->simulate($count);

        return response()->json([
            'status' => 'ok',
            'data' => $result,
            'note' => 'Weighted Round-Robin: server-1(3x) > server-
2(2x) > server-3(1x)',
        ]);
    }

    // POST /api/lb/dispatch - توجيه طلب واحد
    public function dispatch(Request $request): JsonResponse
    {
        $requestId = $request->input('request_id', uniqid('REQ-'));
        $endpoint = $request->input('endpoint', '/api/orders');

        $routed = $this->lb->dispatch($requestId, $endpoint);

        return response()->json(['status' => 'ok', 'data' => $routed]);
    }

    // POST /api/lb/server-down - محاكاة إسقاط خادم (للعرض)
    public function markDown(Request $request): JsonResponse
    {
        $serverId = $request->input('server_id', 'server-3');
        $this->lb->markServerDown($serverId);
    }
}

```

```
        return response()->json(['status' => 'ok', 'message' =>
"{$serverId} marked DOWN. Failover active."]);
    }

    // POST /api/lb/server-up — إعادة الخادم
    public function markUp(Request $request): JsonResponse
    {
        $serverId = $request->input('server_id', 'server-3');
        $this->lb->markServerUp($serverId);
        return response()->json(['status' => 'ok', 'message' =>
"{$serverId} restored."]);
    }
}
```